FastThaiG2P: Lightning-fast Thai Grapheme-to-phoneme Conversion for Voice Agent Pipelines

Charin Polpanumas
AWS
charipol@amazon.com

Abstract

FastThaiG2P provides sub-millisecond Thai grapheme-to-phoneme conversion for text-to-speech pipelines (International Phonetic Alphabet and Kokoro-TTS conventions) using a PyThaiNLP-tokenized, extensible dictionary and normalization rules for common Central Thai speech. The approach achieves an average latency of 0.15 ms per utterance on a benchmark of 27,242 synthetically generated utterances, of which 30% is spent on tokenization, 12% on normalization, and 58% on out-of-vocabulary fallbacks (0.5% OOV rate). To demonstrate its effectiveness, we used FastThaiG2P to phonemize Som-TTS, an open dataset containing 20 hours of grapheme-and-audio pairs, then trained an 82M-parameter StyleTTS 2 model based on a Kokoro-TTS recipe. The resulting model vocalizes intelligible Thai speech suitable for prototyping and development at 0.25 real-time factor (4x real-time) with ONNX inference on CPU.

1 Introduction

Challenges in Thai Phonemization Thai text-to-speech (TTS) systems face a unique set of challenges at the grapheme-to-phoneme (G2P) stage. Unlike languages with relatively transparent orthography, alphasyllabary Thai script exhibits several properties that complicate phonemization: 1) words and sentences are written without explicit boundaries, requiring segmentation as a prerequisite to phonemization 2) Thai is a tonal language with five lexical tones whose surface realization depends on syllable structure, consonant class, vowel length, and final consonant 3) vowels are written as discontinuous graphemes that surround their onset consonant and some are polyfunctional, serving as standalone vowels as well as components of other vowels 4) multiple classes of non-transparent grapheme-to-phoneme mappings–namely leading consonant tone shift, reduced vowels absent from the orthography, context-dependent ligatures, false clusters, and homographs–mean that character-level rules alone are insufficient 5) real-world Thai text contains a dense mixture of numbers, abbreviations, symbols, English loanwords, and code-switched tokens that must be verbalized before phonemization. For real-time voice agent pipelines such as call center agents, conversational AI assistants, and live captioning systems, the G2P stage must be both comprehensive and fast. A latency budget of 500–1,000 ms for the entire TTS pipeline leaves little room for slow preprocessing.

Existing Approaches Several tools provide Thai G2P functionality, spanning rule-based, dictionary-based, and model-based approaches. **TLTK** [1] performs syllable-level G2P via a trigram-based syllable segmenter and rule-based phonological mapping. It handles regular Thai orthography well but relies entirely on rules without a pre-built dictionary, limiting accuracy on irregular words and

loanwords. Its runtime recompiles regex patterns on every call, resulting in high latency for batch processing, up to 2 ms per utterance on average over our 27,242-utterance synthetic dataset. **Epitran** [2] offers rule-based grapheme-to-IPA mapping for 61 languages including Thai via a character map with pre- and post-processing rewrite rules. Its Thai mode reorders leading vowels, applies coda neutralization, and maps consonants/vowels to IPA segments. However, it explicitly discards tone marks without inferring tonal values from syllable structure, requires pre-segmented word input, and does not include word segmentation or text normalization. **thai-g2p** [3] uses a seq2seq, MarianMT-based model [4] trained on Wiktionary data to predict phonemes from Thai words. It requires external word segmentation and does not include text normalization. **CharsiuG2P** [5] is a ByT5-based [6] multilingual neural G2P covering 100 languages including Thai. It achieves a phoneme error rate (PER) of 26.9% and word error rate (WER) of 59% and produces IPA with tonal notation for Thai. However, it requires pre-tokenized word input, has higher inference latency than dictionary lookup due to autoregressive decoding, and does not include text normalization. None of these tools simultaneously provide 1) a large curated IPA dictionary for Thai, 2) comprehensive text normalization for spoken forms, 3) sub-millisecond per-utterance latency, and 4) robust OOV handling, all of which are pre-requisites for inputs to phoneme-based TTS architectures [7] [8].

Phonemization-free Thai TTS Recent grapheme-based TTS models can synthesize Thai speech directly from text without explicit G2P. MMS-TTS [9] provides a lightweight VITS [10] model for Thai but suffers from high error rates (CER 18.28% on Common Voice 13 Thai test split [11]). ThonburianTTS [12] finetunes F5-TTS [13] on 969 hours of Thai speech, achieving CER 8.70% on Common Voice 13 Thai test split. JaiTTS [14] adapts VoxCPM-0.5B [15] for Thai with CER of 1.94% on their internal benchmark. Multilingual architectures including Qwen3-TTS [16], OmniVoice [17], VoxCPM-0.5B [15] and VoxCPM2 [18] support Thai among many languages. However, these models require 336M–2B parameters and GPU inference. For deployment scenarios requiring CPU-only inference, sub-second latency, or minimal memory footprint such as edge devices and cost-sensitive batch pipelines, a phoneme-based approach with a compact acoustic model [10] [7] [19] [8] remains the practical choice thus the necessity of a fast and accurate phonemization front-end.

FastThaiG2P Enables Intelligible, Low-Latency, CPU-based TTS This paper presents FastThaiG2P, an open-source Thai G2P library that bridges the gap between dictionary-based accuracy and the speed required for real-time phoneme-based TTS. The system combines a 62,112-word IPA dictionary with comprehensive text normalization and a rule-based out-of-vocabulary (OOV) fallback, achieving 0.15 ms per utterance on a 27,242-utterance benchmark. Paired with a Thai-finetuned Kokoro-82M [8] checkpoint based on StyleTTS 2 [19], the full pipeline produces intelligible Thai speech at 0.25 real-time factor (RTF) on CPU with a total footprint of 330 MB. **Our contributions are**: 1) a 62,112-word IPA dictionary assembled from Wiktionary, LLM-generated transcriptions validated against phonological rules, and manual overrides 2) a text normalization pipeline covering 15 categories of non-speakable text including numbers, Thai abbreviations, units, symbols, phone numbers, emails, and time patterns 3) up to 15x latency reduction over the TLTK baseline through regex caching and import-time initialization 4) an end-to-end TTS demonstration using FastThaiG2P to train a Thai Kokoro-82M model achieving 0.25 RTF on CPU. The library is released under Apache-2.0 at `github.com/aws/FastThaiG2P`.

2 FastThaiG2P System Design

FastThaiG2P implements a four-stage pipeline, sequentially, text normalization, tokenization, phoneme dictionary lookup, and fallback G2P.

2.1 Text Normalization

The normalizer converts non-vocalizable scripts to vocalizable ones before tokenization. This acts as the deterministic, last line of defense against invalid TTS inputs in case prompting the large language

model (LLM) fails in a cascading voice agent pipeline (automatic speech recognition (ASR) → LLM → TTS). Processing occurs in a fixed order designed to prevent ambiguity:

- Expand maiyamok, the Thai word repetition marker
- Convert Thai numerals to Arabic digits
- Read email addresses
- Read English abbreviations and brand names (transliteration dictionary)
- Read units (kg, km, °C, etc.)
- Read symbols (%, +, ×, etc.)
- Read time patterns (14:30, 23:12, etc.)
- Read phone numbers (digit-by-digit per group)
- Read alphanumeric identifiers (ORD-001)
- Read comma-separated numbers (1,000)
- Read decimal and plain numbers
- Read Thai abbreviations
- Read residual Latin characters

Short numbers (6 digits or shorter) use Thai place-value reading while long numbers (7 digits or longer) are read digit-by-digit, matching the Thai convention for phone numbers, account numbers, and IDs.

2.2 Tokenization

After normalization, the text is segmented into words using PyThaiNLP's 'newmm' (maximum matching) engine [20] with a custom dictionary. The dictionary ('data/dict.txt' and 'data/ipa.json') serves both as a custom word-list for the segmentation and lookup keys for corresponding IPA entries. This coupling ensures every token produced by the word segmenter has an IPA barring OOV tokens.

2.3 Dictionary Construction

The 62,112-entry IPA dictionary was built from three main sources, from highest to lowest merge priorities:

Manual Overrides (`sources/manual_overrides.json`) manually verified IPA transcriptions for words where we could not find references from Wikitionary [21] and/or automated methods fail. This serves as a human-in-the-loop lever for continuous dictionary improvement.

Wiktionary (`sources/wiktionary_ipa.json`) about 13,000 entries extracted from the Thai Wiktionary dump. These follow the English Wiktionary IPA convention for Thai, which served as the reference standard for our transcription format.

LLM Transcription (`sources/generated_ipa.json`) approximately 49,000 entries generated by Claude Opus 4.6 via Amazon Bedrock. Generation was performed in batches of 100 words using a detailed prompt (Figure 5; `scripts/ipa_prompt.txt`) specifying the IPA convention, phonological constraints, and example transcriptions. Each response was validated by checking that every character falls within a 38-codepoint phoneme inventory whitelist (consonants, vowels, Chao tone letters, combining diacritics, and separators); entries with out-of-inventory characters or missing slash delimiters were rejected and logged to `generated_ipa_invalid.json`.

Before batch generation, the prompt was validated against 500 random Wiktionary entries as a validation set (`scripts/validate_prompt.py`). With TTS as the objective, errors were classified

as non-fatal (tone mismatch, vowel length, diphthong marker, coda variant) or fatal (wrong syllable count, wrong initial consonant or vowel quality). Pali/Sanskrit loanwords with irregular readings were excluded from the fatal count since Wiktionary handles them directly. The final prompt has 78.8% exact match rate and 1.8% fatal error rate on the validation set.

2.4 OOV Fallback

Words not found in the dictionary receive IPA transcription via a rule-based fallback vendored from TLTK [1]. The fallback segments the word into syllables using trigram statistics (‘data/fallback/sylseg.3g‘), maps each syllable to a romanized pronunciation using consonant class, vowel pattern, and tone rules, then converts the romanization to our IPA convention (Chao tone letters, aspiration, unreleased stops). The fallback always produces phonologically valid Thai IPA but may not match the conventional pronunciation of irregular words (loanwords, Pali/Sanskrit compounds with silent letters). Frequently encountered OOV words should be manually added to the dictionary.

Regex Cache Optimization The original TLTK implementation recompiles regular expressions on every function call. The `sylparse` function iterates over hundreds of syllable pattern regexes, calling `re.match(pattern_string, ...)` at every character position of the input word. Python’s internal regex cache, a Least Recently Used (LRU) cache limited to 512 entries, overflows when the number of unique patterns exceeds this limit, causing repeated recompilation. In the vendored version, we introduced an explicit unbounded regex cache at all three hot loops in the fallback code: syllable parsing, syllable enumeration, and rule loading. Combined with pre-loading the syllable rules at import time rather than on first call, this reduced the overall G2P latency from approximately 2 ms per utterance to 0.15 ms per utterance, up to 15x improvement. The majority of the savings come from eliminating redundant regex compilation in the fallback path, which previously dominated wall-clock time even though it was invoked on a minority of tokens.

2.5 IPA Convention

We follow the English Wiktionary Thai IPA transcription standard [22] using Chao tone letters (Figure 1). The inventory uses 38 phoneme-related codepoints.

Feature	Notation	Example
Tones	Chao tone letters	˧ mid, ˨˩ low, ˩˧ falling, ˥ high, ˨˩˦ rising
Syllable boundary	.	sa˨˩.wat̚˨˩.diː˧
Vowel length	ː	aː (long) vs a (short)
Aspiration	ʰ	kʰ (aspirated) vs k (unaspirated)
Unreleased stops	̚	t̚, p̚, k̚
Affricates	t͡ɕ , t͡ɕʰ	จ, ช
Diphthongs	̯ (non-syllabic)	iạ̯, uạ̯, ɯạ̯
Glottal stop	ʔ	ɐ (initial)
Word boundary	space	between /.../ groups

Figure 1: IPA Convention

IPA-to-Kokoro Mapping Kokoro-82M was pretrained on English and does not natively support Thai. Its phoneme vocabulary includes four intonation markers used for English prosody; however Thai requires five tonally distinct markers. A naive mapping would force a merger between high and rising tones, which are phonemically contrastive. We repurposed an unused token ID 170 as the high

tone marker, yielding the following five-tone mapping. We also stripped diacritics that only exist in IPA as well as substitute affricates and g character with the corresponding Kokoro equivalents (See Figure 2).

Thai tone	IPA (Chao)	Kokoro token
Mid	˧	→
Low	˨˩	↓
Falling	˥˩	↘
High	˦˥	↑ (repurposed unused token ID 170)
Rising	˩˩˦	↗
Affricates		
t͡ɕ		tɕ (precomposed)
t͡ɕʰ		tɕʰ (precomposed)
Stripped diacritics		
˺ (unreleased stop)		removed
˕ (non-syllabic)		removed
◌͡◌ (tie bar)		removed
Character normalization		
g (ASCII)		g (IPA)

Figure 2: IPA-to-Kokoro Mapping

3 Latency Profiling

Latency was measured on a corpus of 27,242 synthetically generated Thai utterances (`data/synthetic/utterances.jsonl`) representing conversational text, numbers, abbreviations, and domain-specific vocabulary typical of voice agent interactions. Profiling was performed using `scripts/profile_g2p.py` which measures end-to-end latency across all utterances and per-component timing with cProfile instrumentation on a single-threaded CPU setup with Python 3.11. Despite only 0.5% of tokens being OOV, the fallback dominates wall-clock time (58%) because trigram-based syllable parsing is orders of magnitude more expensive per-token than a hash lookup. The pipeline exhibits predictable latency with no neural network inference, no disk input-output per call, and no per-call memory allocation. See Figure 3.

4 Text-to-Speech Demonstration

To validate that FastThaiG2P enables a minimally viable Thai TTS system with CPU-only inference, we trained a Thai-finetuned Kokoro-82M checkpoint end-to-end. We used Kokoro-82M [8], an 82-million-parameter architecture based on StyleTTS 2 [19]. The model was Thai-finetuned using the kikiri-tts training recipe [23], which implements a two-stage training process: stage 1 for text-to-mel alignment and duration prediction, and stage 2 for adversarial training with a multi-scale discriminator. The model takes Kokoro-format phoneme sequences as input and produces 24 kHz audio. A voicepack vector conditions the model on a target speaker identity. We used Som-TTS [24], an open Thai TTS dataset containing approximately 20 hours of single-speaker recordings with aligned Thai text transcripts. FastThaiG2P was used to phonemize all transcripts, converting Thai

Metric	Value
Average latency	0.15 ms/utterance
Throughput	6,600 utterances/second

Stage breakdown (all 27,242 utterances):

Stage	Time share	Notes
Tokenization	30%	PyThaiNLP newmm dictionary matching
Normalization	12%	Regex-based text transformations
Fallback G2P	58%	Only invoked for OOV tokens (0.5% OOV rate)
Dictionary lookup	< 1%	O(1) hash table, negligible

Figure 3: Latency Profiling Result

graphemes to the Kokoro phoneme format via the IPA intermediate representation. For deployment without PyTorch, the trained checkpoint was exported to ONNX format, outputting raw audio samples plus frame durations for boundary token trimming.

The resulting model produces intelligible Thai speech with recognizable tonal patterns and natural rhythm for short to medium utterances at RTF 0.25 (4x real-time) on a single-threaded CPU inference. The total memory footprint is about 330 MB (See Figure 4). Audio quality is suitable for prototyping and development. The TTS module includes a post-processing step that removes static/noise generated during the beginning-of-sentence (BOS) and end-of-sentence (EOS) pad token windows using voicing detection to distinguish genuine speech onset from artifacts in the boundary regions.

Metric	Value
Real-time factor (RTF)	0.25 (4× real-time)
Inference backend	ONNX, CPU only
Sample rate	24 kHz
Checkpoint size	330 MB (model + voicepack + config)
Max input	510 phonemes

Figure 4: TTS Inference Result

5 Discussion and Future Work

Dictionary Coverage The 62k-word dictionary covers standard Central Thai well but has gaps in regional vocabulary, recent loanwords, slang, and code-switched Thai-English text. We expect to periodically update the dictionary and allows seamless manual overrides for each use case.

Fallback Quality The rule-based fallback produces phonologically valid IPA but may err on irregular words (silent letters in Pali/Sanskrit compounds, non-standard loanword pronunciations). A hybrid approach combining rules with a small neural model could improve OOV handling.

Tone Accuracy Thai tone assignment from orthography is largely rule-governed but has exceptions such as tone-mark elision in certain compounds, dialectal variation. The current system follows standard Central Thai rules; regional variants are not modeled.

TTS Quality Evaluation No formal MOS study has been conducted. A perceptual evaluation with native Thai speakers would quantify quality and enable comparison against commercial Thai TTS systems.

Future Directions We see FastThaiG2P as an enabler for increasingly more potent phoneme-based, CPU-only TTS systems that power hybrid voice agents, agentic systems which leverage frontier models to perform complex tasks on AWS Bedrock and Sagemaker while outsourcing simpler tasks such as ASR and TTS to local devices.

6 Appendix

6.1 Prompt for IPA Transcription

References

[1] Wirote Aroonmanakun and Attapol Thamrongrattanarit. Tltk: Thai language toolkit. `https://github.com/attapol/tltk`, 2018.

[2] David R Mortensen, Siddharth Dalmia, and Patrick Littell. Epitran: Precision g2p for many languages. In *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC 2018)*, 2018.

[3] Wannaphong Phatthiyaphaibun. thai-g2p-v2: Thai grapheme-to-phoneme, 2020.

[4] Marcin Junczys-Dowmunt, Roman Grundkiewicz, Tomasz Dwojak, Hieu Hoang, Kenneth Heafield, Tom Neckermann, Frank Seide, Ulrich Germann, Alham Fikri Aji, Nikolay Bogoychev, et al. Marian: Fast neural machine translation in c++. In *Proceedings of ACL 2018, system demonstrations*, pages 116–121, 2018.

[5] Jian Zhu, Cong Zhang, and David Jurgens. Byt5 model for massively multilingual grapheme-to-phoneme conversion. 2022.

[6] Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. Byt5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10:291–306, 2022.

[7] M Hansen. Piper: A fast, local neural text to speech system. *Piper: A fast, local neural text to speech system*, 2023.

[8] hexgrad. Kokoro-82m: Open-weight tts model. `https://huggingface.co/hexgrad/Kokoro-82M`, 2024.

[9] Vineel Pratap, Andros Tjandra, Bowen Shi, Paden Tomasello, Arun Babu, Sayani Kundu, Ali Elkahky, Zhaoheng Ni, Apoorv Vyas, Maryam Fazel-Zarandi, et al. Scaling speech technology to 1,000+ languages. *Journal of Machine Learning Research*, 25(97):1–52, 2024.

[10] Jaehyeon Kim, Jungil Kong, and Juhee Son. Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech. In *International conference on machine learning*, pages 5530–5540. PMLR, 2021.

[11] Rosana Ardila, Megan Branson, Kelly Davis, Michael Kohler, Josh Meyer, Michael Henretty, Reuben Morais, Lindsay Saunders, Francis Tyers, and Gregor Weber. Common voice: A massively-multilingual speech corpus. In *Proceedings of the twelfth language resources and evaluation conference*, pages 4218–4222, 2020.

[12] Thura Aung, Panyut Sriwirote, Thanachot Thavornmongkol, Knot Pipatsrisawat, Titipat Achakulvisut, and Zaw Htet Aung. Thonburiantts: Enhancing neural flow matching models for authentic thai text-to-speech. In *2025 20th International Joint Symposium on Artificial Intelligence and Natural Language Processing (iSAI-NLP)*, pages 1–6. IEEE, 2025.

[13] Yushen Chen, Zhikang Niu, Ziyang Ma, Keqi Deng, Chunhui Wang, JianZhao JianZhao, Kai Yu, and Xie Chen. F5-tts: A fairytaler that fakes fluent and faithful speech with flow matching. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6255–6271, 2025.

[14] Jullajak Karnjanaekarin, Pontakorn Trakuekul, Narongkorn Panitsrisit, Sumana Sumanakul, Vichayuth Nitayasomboon, Nithid Guntasin, Thanavin Denkavin, and Attapol T Rutherford. Jaitts: A thai voice cloning model. *arXiv preprint arXiv:2604.27607*, 2026.

[15] Yixuan Zhou, Guoyang Zeng, Xin Liu, Xiang Li, Renjie Yu, Ziyang Wang, Runchuan Ye, Weiyue Sun, Jiancheng Gui, Kehan Li, Zhiyong Wu, and Zhiyuan Liu. Voxcpm: Tokenizer-free tts for context-aware speech generation and true-to-life voice cloning. *arXiv preprint arXiv:2509.24650*, 2025.

[16] Hangrui Hu, Xinfa Zhu, Ting He, Dake Guo, Bin Zhang, Xiong Wang, Zhifang Guo, Ziyue Jiang, Hongkun Hao, Zishan Guo, et al. Qwen3-tts technical report. *arXiv preprint arXiv:2601.15621*, 2026.

[17] Han Zhu, Lingxuan Ye, Wei Kang, Zengwei Yao, Liyong Guo, Fangjun Kuang, Zhifeng Han, Weiji Zhuang, Long Lin, and Daniel Povey. Omnivoice: Towards omnilingual zero-shot text-to-speech with diffusion language models. *arXiv preprint arXiv:2604.00688*, 2026.

[18] Yixuan Zhou, Guoyang Zeng, Xin Liu, Xiang Li, Renjie Yu, Jiancheng Gui, Jiaheng Wu, Ziyang Wang, Xudong Shen, Runchuan Ye, Zhisheng Zhang, Jiuyang Zhou, Bingsong Bai, Weiyue Sun, Mengyuan Deng, Qundong Shi, Zhiyong Wu, and Zhiyuan Liu. Voxcpm2 technical report. *arXiv preprint arXiv:2606.06928*, 2026.

[19] Yinghao Aaron Li, Cong Han, Vinay Raghavan, Gavin Mischler, and Nima Mesgarani. Styletts 2: Towards human-level text-to-speech through style diffusion and adversarial training with large speech language models. *Advances in neural information processing systems*, 36:19594–19621, 2023.

[20] Wannaphong Phatthiyaphaibun, Korakot Chaovavanich, Charin Polpanumas, Arthit Suriyawongkul, Lalita Lowphansirikul, Pattarawat Chormai, Peerat Limkonchotiwat, Thanathip Suntorntip, and Can Udomcharoenchaikit. Pythainlp: Thai natural language processing in python. In *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS 2023)*, pages 25–36, 2023.

[21] Tatu Ylonen. Wiktextract: Wiktionary as machine-readable structured data. In *Proceedings of the Thirteenth Language Resources and Evaluation Conference*, pages 1317–1325, 2022.

[22] Wikipedia contributors. Wikipedia:Manual of Style/Pronunciation — Wikipedia, the free encyclopedia. `https://en.wikipedia.org/wiki/Wikipedia:Manual_of_Style/Pronunciation#Entering_IPA_characters`, 2026. [Online; accessed 2026-07-01].

[23] semidark. kikiri-tts: Training recipe for fine-tuning kokoro-82m on a new language. `https://github.com/semidark/kikiri-tts`, 2026. [Online; accessed 26-July-2026].

[24] PyThaiNLP. Som TTS dataset: Open Data Thai TTS, July 2026.

```
You are a Thai linguistics expert. Convert Thai words to IPA phonemic transcription following these exact conventions:

## Format
- Enclose in forward slashes: /transcription/
- Separate syllables with a dot: /syl.la.ble/
- One word per line, format: word/ipa/

## Consonants
Initial consonants:
  p (ป), pʰ (พ,ผ,ภ), b (บ)
  t (ต,ฏ), tʰ (ท,ถ,ฑ,ฒ,ธ), d (ด,ฎ)
  k (ก), kʰ (ข,ฃ,ค,ฅ,ฆ), ʔ (อ as initial, vowel-initial syllables)
  t͡ɕ (จ), t͡ɕʰ (ช,ฉ,ฌ)
  f (ฝ,ฟ), s (ส,ซ,ศ,ษ), h (ห,ฮ)
  m (ม), n (น,ณ), ŋ (ง)
  l (ล,ฬ), r (ร), w (ว), j (ย,ญ)

IMPORTANT: For จ always use t͡ɕ (with tie bar‿). For ช,ฉ,ฌ always use t͡ɕʰ. NEVER use tʃ or tɕ without the tie bar.

## Final Consonants (Coda)
Thai has only 9 possible coda sounds. ALL written consonants in coda position map to one of these:

  p̚ ← บ,ป,พ,ฟ,ภ
  t̚ ← ต,ถ,ท,ธ,ฒ,ฑ,ส,ศ,ษ,ซ,ฎ,ฏ,ฐ,ฒ,ฌ
  k̚ ← ก,ข,ค,ฆ
  m ← ม
  n ← น,ณ,ญ,ร,ล,ฬ
  ŋ ← ง
  j ← ย,ญ (when part of vowel)
  w ← ว (when part of vowel)
  ʔ ← glottal stop (see below)

EXCEPTION for loanwords: Foreign words that are commonly read with preserved final /s/ or /l/ keep those sounds. Examples: ปารีส /paːɹ.riː

## Glottal Stop (ʔ) as Final
MANDATORY: Any syllable with a SHORT vowel and NO written final consonant ends with ʔ.
- กะปิ → /kaˈɹ.piʔˈɹ/
- ผุ → /pʰuʔˈɹ/
- ขยะ → /kʰaˈɹ.jaʔˈɹ/
- จะ → /t͡ɕaʔˈɹ/
- ธาตุ → /tʰaːt̚ˈɹ.tuʔˈɹ/

Do NOT add ʔ if vowel is long or there is a final consonant.

## Vowels
Short: a, i, u, e, ɛ, o, ɔ, ɤ, ɯ
Long: aː, iː, uː, eː, ɛː, oː, ɔː, ɤː, ɯː

Diphthongs — mark second element with ̯ (U+032F):
  ia̯ (เ◌ีย), ɯa̯ (เ◌ือ), ua̯ (◌ัว, ◌วC clusters)
CRITICAL: ALWAYS write ̯ — "ua̯" not "ua", "ia̯" not "ia".

## Vowel Length — CRITICAL RULES

### Always SHORT (no ː):
- ◌ะ, ◌ั◌ : a
- ◌ิ : i
- ◌ุ : u
- เ◌ะ : e
- แ◌ะ : ɛ
- โ◌ะ : o
- เ◌าะ : ɔ
- เ◌อะ : ɤ
- ◌ึ : ɯ
- เ◌ือ : ɯa̯ (short diphthong)
- เ◌ีย : ia̯ (short diphthong)
- ◌ำ : am (short a + m)
- **◌อ◌ in CLOSED syllables** (with a final consonant): ɔ (SHORT!)
  Examples: ห่อง /hɔŋˈɹ/, ล่อง /lɔŋˈɹ/, ค่อน /kʰɔnˈɹ/, ซ่อม /sɔmˈɹ/, หอย /hɔjˌɹˌɹ/, ค่อย /tɔjˈɹ/
- **แ◌ with ◌็ (mai taikhu)**: ɛ (short) — แซ็ง /kʰɛŋˌɹˌɹ/
- **แ◌ in closed syllable when preceded by consonant cluster**: ɛ (short) — แสป /hɛp̚ˈɹ/
- **เ◌ือ◌ (sara oe with final consonant)**: ɤ (short) — เกิด /kɤːt̚ˈɹ/ WAIT — actually เ◌ือ◌ is complex:
  - เกิด = /kɤːt̚ˈɹ/ (LONG — this is sara oe, which IS long)
  - Correction: เ◌ือ◌ before a single final = LONG ɤː. Only short when in เ◌ือ◌ะ pattern.

Let me correct: เ◌ือ◌ (without ะ) = LONG ɤː. Example: เกิด /kɤːt̚ˈɹ/, เกิน /kɤːnˈɹ/

### Always LONG (with ː):
- ◌า : aː
- ◌ี : iː
- ◌ู : uː
- เ◌ (open or before single final): eː
- แ◌ (without ะ or ◌็): ɛː
- โ◌ (without ะ): oː
- ◌อ in OPEN syllables (no final): ɔː — พ่อ /pʰɔːˈɹ/, คอ /kʰɔːˌɹ/
- เ◌ือ◌ (sara oe, no ะ): ɤː — เกิด /kɤːt̚ˈɹ/
- ไ◌, ใ◌ : aːj (long a + j)
- เ◌า : aw (short a!) — เช้า /t͡ɕʰawˌɹ/, เช้า /kʰawˈɹ/

WAIT — เ◌า (sara ao) = SHORT a + w. This is NOT long. เช้า = /t͡ɕʰawˌɹ/.

### Tricky cases:
- เ◌า (sara ao) = /aw/ (short a + w final): เช้า /t͡ɕʰawˌɹ/, เช้า /kʰawˈɹ/, เบา /bawˌɹ/
```

Figure 5: Prompt for IPA Transcription